CSE IIT Bombay Programming Test - 9th May 2025 (Batch A)

Batch A - Q1

Armstrong Number

An Armstrong number is a number that is equal to the sum of its own digits, each raised to the power of its number of digits. For example:

- 153 is an Armstrong number because its number of digits is 3, and $1^3 + 5^3 + 3^3 = 153$
- 9474 is an Armstrong number because its number of digits is 4, and $9^4 + 4^4 + 7^4 + 4^4 = 9474$

Write a C/C++ program that accepts a number N representing the number of integers to check. Then, for each of the N integers, determine whether it is an Armstrong number or not. If it is an Armstrong number, print "y", otherwise, print the sum calculated.

Input Format:

- The first line contains a single integer N, the number of integers to check.
- The next line contains N integers, each separated by a space.

Output Format:

For each of the N integers:

- If the number is an Armstrong number, print "y".
- Otherwise, print the sum of the digits, each raised to the power of the number of digits in the number.
- After each output, print a space.

Assumptions on Input:

- The value of N entered will be between 1 and 1000 both inclusive
- Each of the N integers will be between 1 and 1000000

Sample Input:

```
3
153 9474 123
```

Sample Output:

```
y y 36
```

Practice Testcase

Input	Output
3 5 88999 407	y 242683 y
4 153 370 2646 11531	y y 2864 3371
1 9474	y

Batch A - Q2

Compress string

Write a C/C++ program that accepts a string and prints the string with contiguous repeated characters replaced by the character followed by the count. You must consider the case of the character. i.e. ‘A’ is different than ‘a’.

For example:

- "AAAABBBaaCCAAaa" should be converted to "A4B3a2C2A2a2".

Input Format:

- The first line contains a single string

Output Format:

- The output should be a string where each contiguous sequence of repeated characters is replaced by the character followed by its count.

Assumptions on Input:

- The input string will only contain valid lowercase or uppercase alphabetical characters.
- The length of the input string will be between 1 and 1000 characters.

Sample Input:
AAAABBBCC
Sample Output:
A4B3C2

Practice Testcase

Input	Output
KeePer	K1e2P1e1r1
apple	a1p2l1e1
AAAABBBaaCCAAaa	A4B3a2C2A2a2

Batch A - Q3

Valleys in a matrix

Write a C/C++ program to accept two integers M and N as input from the user. These denote the dimensions of a matrix. Then accept the elements of the matrix. Count and print the total number of valleys in the matrix. A cell is considered a valley if it is strictly less than all of its adjacent (neighbour) cells (horizontal and vertical, and not diagonal). For the cell `(i, j)`, the neighbours or adjacent cells are `(i-1, j)`, `(i+1, j)`, `(i, j-1)`, `(i, j+1)`. You must take care if `i, j` is at the boundary.

Example: Consider a 4 x 4 Matrix
9 8 9 0
7 4 6 8
9 7 2 5
3 5 9 5
The number of valleys is 4 at positions (0, 3), (1, 1), (2, 2), (3, 0)

Element	Position `(i, j)`	Adjacent Elements
0	0, 3	9, 8
4	1, 1	8, 7, 7, 6
2	2, 2	6, 7, 9, 9
3	3, 0	9, 5

Note that the element 5 at position (3,3) is **not** a valley because, although it is strictly less than the adjacent element 9, it is **not** strictly less than the adjacent element 5.

Input Format:

- The first line contains two integers M and N, representing the number of rows and columns of the matrix, each separated by a space.
- The next M lines contain N integers representing the elements of the matrix. The elements are separated by a space.

Output Format:

- An integer representing the number of valleys in the matrix.

Assumptions on Input:

- The value of M and N entered by the user will be between 1 and 100
- The matrix elements entered by the user will be between -1000 to 1000.

Practice Testcase

Input	Output	Explanation The positions at which the valleys are
4 4 9 8 9 0 7 4 6 8 9 7 2 9 3 5 9 5	5	0, 3 1, 1 2, 2 3, 0 3, 3
4 4 9 6 9 9 7 1 6 8 9 9 2 9 9 5 9 3	4	1, 1 2, 2 3, 1 3, 3
6 5 10 9 8 9 10 9 -3 7 -2 9 10 8 10 7 10 8 5 -5 8 9 7 6 7 9 8 -6 7 9 8 10	6	1, 1 1, 3 3, 2 4, 4 5, 0 5, 3

Batch A - Q4

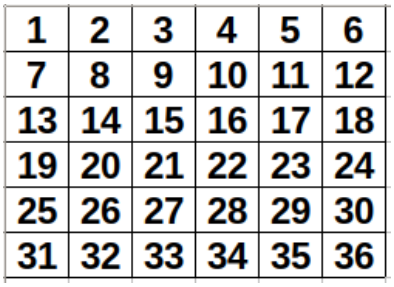
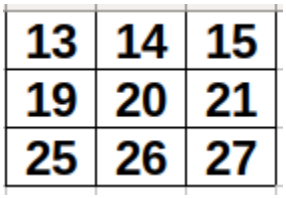
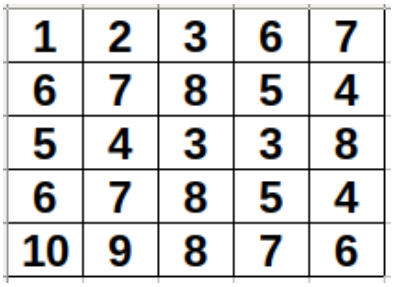
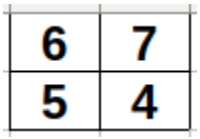
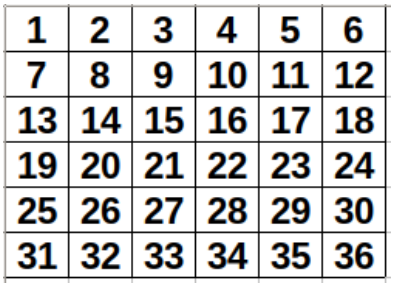
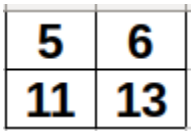
Sub-Matrix

Write a C/C++ program to accept an integer M followed by the M x M integer matrix A. Then accept N followed by N x N integer matrix B. Your task is to determine if B is a sub-matrix of A. In other words, you must determine if there is some row `i` and column `j` in A such that if the "top left" of B is "pasted" at the `(i, j)` element of A, then all the entries of B align with corresponding entries of A. Print the value of `i` and `j` (first occurrence going row first).

If B is not a submatrix of A, you should print `-1`. Otherwise print `i` and `j` (space separated) in the same line, such that B is a submatrix of A if we start from row `i` and column `j` of A (both of which vary from 0 through n - 1). If B matches at multiple starting points in A, print the value of `(i, j)` such that `i` is as small as possible, and for entries having that common value of `i`, the column `j` is as small as possible.

Go through the examples given.

Matrix A	Matrix B	Explanation
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36	13 14 15 19 20 21 25 26 27	Matrix B is a sub-matrix of Matrix A, starting at top left index `2,0`
1 2 3 6 7 6 7 8 5 4 5 4 3 3 8 6 7 8 5 4 10 9 8 7 6	6 7 5 4	Matrix B is a sub-matrix of Matrix A at two locations whose top-left corners are: `0,3 and 1,0` We choose `0,3` as row 0 entry appears before row 1 entry
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36	13 14 33 19 20 21 25 26 27	Matrix B is not a sub-matrix of Matrix A

Matrix A	Matrix B	Explanation
		Matrix B is a sub-matrix of Matrix A, starting at top left index `2,0`
		Matrix B is a sub-matrix of Matrix A at two locations whose top-left corners are: `0,3 and 1,0` We choose `0,3` as row 0 entry appears before row 1 entry
		Matrix B is not a sub-matrix of Matrix A

Input Format:

- The first line contains an integer M
- The next M lines contain M integers, each separated by a single space, that represent the elements of the MxM matrix
- The next line contains an integer N
- The subsequent N lines contain N integers, each separated by a single space, that represent the elements of the NxN matrix

Output Format:

- Two integers that represent the index or -1

Assumptions on Input:

- The value of M and N entered by the user will be between 1 and 100
- The matrix elements entered by the user will be between -1000 to 1000.
- Assume that the value of N will always be less than or equal to M

Practice Test cases

Input	Output
6 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 3 13 14 15 19 20 21 25 26 27	2 0
6 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 3 13 14 33 19 20 21 25 26 27	0 3
10 2 -9 -8 -2 0 -1 -2 -4 -7 2 -3 -9 -2 7 5 2 4 -1 -9 6 -5 -3 -3 9 4 7 -8 -9 -2 -2 -7 1 3 -4 5 8 9 -4 -9 9 -5 5 -9 1 7 -7 7 2 -2 0 9 -2 1 -4 1 -7 -3 2 -8 -2 -5 0 0 8 1 0 5 -5 -4 7 1 0 -4 2 -3 1 0 -2 -3 0 -7 -2 -6 0 7 -1 -3 0 9 9 -1 7 0 6 -9 -6 -6 -1 6 8 1 3	3 2

Batch A - Q5

Count Binary

Write a C/C++ program to recursively count the number of binary strings of length `N` containing exactly `K` ones such that no two ones are adjacent to one another.

Example: If N = 4 and K = 2, then the desired number of binary strings will be 3. These strings are:

1010
0101
1001

Input Format:

- The first line contains two positive integers **N** and **K**, separated by a space

Output Format:

- A single integer representing the number of binary strings of length n containing exactly k ones with no two ones adjacent to one another.

Assumptions on Input :

- Assume that the value of N entered by the user will be between 1 and 10, both inclusive
- Assume that the value of K entered by the user will be between 1 and 10, both inclusive
- If it is not possible to construct a binary string of this sort, print 0.

Practice Testcases :

Input	Output
4 2	3
5 3	1
6 5	0

CSE IIT Bombay Programming Test - 9th May 2025 (Batch B)

Batch B - Q1

Disarium Number

A Disarium number is a number that is equal to the sum of its digits, each raised to the power of its respective position in the number. For example:

- 135 is a Disarium number because 1 is at position 1, 3 is at position 2, and 5 is at position 3 and $1^1 + 3^2 + 5^3 = 135$
- 89 is a Disarium number because: $8^1 + 9^2 = 89$

Write a C/C++ program that accepts a number N representing the number of integers to check. Then, for each of the N integers, determine whether it is a Disarium number or not. If it is a Disarium number, print "y", otherwise, print the sum calculated.

Input Format:

- The first line contains a single integer N, the number of integers to check.
- The second line contains N integers, each separated by a space.

Output Format:

For each of the N integers:

- If the number is a Disarium number, print "y".
- Otherwise, print the sum of the digits, each raised to the power of its respective position in the number.
- After each output, print a space.

Assumptions on Input:

- The value of N entered will be between 1 and 1000 (both inclusive).
- Each of the N integers will be between 1 and 1000000.

Sample Input:

3
135 89 123

Sample Output:

y y 32

Practice Testcase

Input	Output
3 5 88999 518	y 66411 y
4 135 89 2646 11531	y y 1398 209
1 175	y

Batch B - Q2

Substring positions

Write a C/C++ program to accept two lowercase strings A and B, and find all starting positions in string A where the entire string B appears in string A as a sub-string. Note that the positions are counted using 0-based indexing i.e. first character of string A is at position 0. Print all positions separated by a single space.

E.g.

```
A: thecatinthehatwearshisownhat
B: hat
hat occurs at positions 11 and 25
```

Input Format:

- The first line contains the first string.
- The second line contains the second string.

Output Format:

- Integers that denote the positions at which the string B appears in string A, each separated by a single space.

Assumptions on Input:

- The input strings will contain only valid lowercase English letters.
- The length of each string will be between 1 and 1000 characters.
- The length of string B will always be less than or equal to the length of string A
- The string B will occur at least once in string A

Practice Testcase

Input	Output
thecatinthehatwearshisownhat hat	11 25
thequickbrownfoxjumpesoverthelazydog o	10 14 22 34
waterbottle water	0

Batch B - Q3

Saddle Point in a Matrix

Write a C/C++ program to accept M and N as input from the user. These denote the dimensions of a matrix. Then accept the elements of the matrix, find, and print the saddle point in it. A cell is considered a saddle point if it is the smallest (strictly) in its entire row and the largest (strictly) in its entire column.

Example: Consider a 3 x 3 matrix
1 2 3
4 5 6
7 8 9
The saddle point is 7 as it is the smallest element in its row and the largest in its column.

Input Format:

- The first line contains two integers M and N, representing the number of rows and columns of the matrix separated by a space.
- The next M lines contain N integers representing the elements of the matrix. The elements are separated by a space.

Output Format:

- An integer representing the saddle point in the matrix

Assumptions on Input:

- The value of M and N entered by the user will be between 1 and 100
- The matrix elements entered by the user will be between -1000 to 1000.
- There will be exactly one saddle point in the matrix

Sample Input:

```
4 5
99 86 82 85 83
34 56 34 11 78
31 30 39 39 34
85 86 80 88 84
```

Sample Output:

82

Practice Testcase

Input	Output
3 3 1 30 35 10 25 40 20 32 33	20
4 5 50 60 65 70 75 55 62 68 74 78 57 63 69 72 76 59 64 66 71 77	59
6 5 -10 20 30 25 15 -15 22 35 18 12 -20 18 28 21 16 -5 24 32 27 19 -100 19 100 26 11 -12 23 29 20 13	-5

Batch B - Q4

Moving Sum

Write a C/C++ program to accept M and N, which represent the dimensions of a matrix A, followed by the elements of the matrix. Then accept two integers, P and Q, representing the size of a submatrix to be considered for computation.

Determine every valid sub-matrix of size PxQ in the matrix A. For each of these submatrices compute the sum of its elements and print it. You must scan the elements (top-left to bottom-right i.e. from 0,0 to M-1, N-1).

Consider an example of a 4x4 Matrix

```
11 12 13 14
15 16 17 18
19 20 21 22
23 24 25 26
```

Assume that P = 3 and Q = 3

All the valid P x Q (3 x 3) submatrix are

Original Matrix with valid PxQ submatrix highlighted	11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26	11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26	11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26	11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26
Valid elements of PxQ submatrix	11 12 13 15 16 17 19 20 21	12 13 14 16 17 18 20 21 22	15 16 17 19 20 21 23 24 25	16 17 18 20 21 22 24 25 26
Sum of the elements	144	153	180	189

Input Format:

- The first line contains two integers, M and N
- The next M lines contain N integers, each separated by a single space, that represent the elements of the MxN matrix
- The next line contains two integers, P and Q

Output Format:

- Integers that represent the sum

Assumptions on Input:

- The value of M and N entered by the user will be between 1 and 100
- The matrix elements entered by the user will be between -1000 to 1000.
- Assume that the value of P will always be less than or equal to M and the value of Q will always be less than or equal to N.

Practice Testcases

Input	Output
3 3 5 12 19 8 3 16 1 14 7 2 2	28 50 26 40
4 3 45 72 60 33 80 47 65 39 55 70 31 76 4 3	673
5 4 -10 34 99 -87 12 -45 67 0 73 -64 23 18 -100 100 -12 45 9 -77 33 50 3 2	0 114 120 -24 69 141 -59 3 157

Batch B - Q5

Tower Blocks

Assume you are given blocks of size 1, 2 and 3. Write a C/C++ program to recursively count the number of ways in which a tower of size N can be built with these blocks, such that exactly K of the blocks are of size 1.

Example: If N = 4 and K = 2, then there are three possible ways in which the tower can be built. If L1, L2, L3 represent the lego blocks of sizes 1, 2, 3 respectively, then the tower of size 4 can be built using the following block arrangements:

L1, L1, L2
L1, L2, L1
L2, L1, L1

Input Format

- A line containing two positive integers **N** and **K**, separated by a space

Output Format

- An integer representing the count of valid ways to build a tower.

Assumptions on Input

- Assume that the value of N entered by the user will be between 1 and 15, both inclusive
- Assume that the value of K entered by the user will be between 1 and 10, both inclusive
- If it is not possible to construct a tower, print 0.

Practice Testcases :

Input	Output
4 2	3
5 1	3
6 2	6